Computer Vision Group Project

ARI3129

Assignment Report

Daniel Pace, Amy Spiteri, Ellyn Rose Debrincat, Joachim Grech

January 31, 2026

Contents

GitHub repository link: `https://github.com/spiteriamy/ari3129-cv-group-project`

1 Introduction

The aim of this project was to implement and evaluate various object detection algorithms on a custom dataset. The project was split up into the following sections:

- Building a dataset:
    1. Image collection,
    2. Image cleaning,
    3. Image annotation,
    4. Data analysis, and
    5. Data preprocessing.
- Object detection/ localization:
    1. Implementation of two object detectors to detect sign type,
    2. Implementation of one object detector to detect a specific sign attribute, and
    3. Evaluation of the trained detectors, including metrics and visualizations.
    4. Analysis of input image, supporting the maintenance and monitoring of traffic signs.

The following steps were taken to ensure an equal and fair distribution of work among all group members:

- For task 1, each group member contributed a similar number of annotated images to the dataset.
- For task 2, each team member was assigned one YOLO-based detector and one COCO-based detector to implement and evaluate.

2 Background on the techniques used

2.1 Faster R-CNN

Faster R-CNN [2] represents a significant advancement in object detection by unifying the region proposal step with the detection network, thereby eliminating the computational bottleneck of external algorithms like Selective Search used in R-CNN [4] and Fast R-CNN [3].

The core innovation is the Region Proposal Network (RPN), a fully convolutional network that shares features with the detection head. The RPN slides a window over the convolutional feature map generated by a backbone such as ResNet [5] to simultaneously predict object bounds and objectness scores at each position. To handle multi-scale detection, it utilizes anchor boxes of various scales and aspect ratios. The entire system is trained end-to-end using a multi-task loss function that combines classification and bounding box regression objectives.

2.2 RF-DETR

RF-DETR [6] is a state-of-the-art real-time object detection and instance segmentation model developed by Roboflow. It belongs to the DETR (DEtection TRansformer) framework for object detection systems, which combines a common CNN backbone with a transformer encoder-decoder architecture [7]. RF-DETR advances this line of work by improving the trade-off between high accuracy and fast inference that was observed in previous works, making it suitable for both high performance and real-time applications. The model integrates internet-scale pretraining to overcome the limitations of previous specialist detectors that overfit to specific benchmarks like COCO [6].

RF-DETR makes use of a pre-trained DINOv2 [8] vision transformer backbone. The use of DINOv2 improves detection accuracy on smaller datasets, however the number of layers in its encoder also contributes to higher latency. However, this is compensated for through the use of NAS. A key contribution from the RF-DETR research is the use of Neural Architecture Search (NAS) to explore and evaluate many model

configurations to find the optimal accuracy-latency Pareto curve for the different configurations. This allows the model to train many different configurations in parallel which enables RF-DETR to adapt to various datasets and hardware platforms without re-training.

RF-DETR is evaluated on the Microsoft COCO and Roboflow100-VL (RF100-VL) datasets. The authors in [6] demonstrate that RF-DETR consistently outperforms other models and achieves state-of-the-art results in both object detection and instance segmentation. RF-DETR sets a new standard as the first real-time object detector to exceed 60 AP on the COCO dataset.

2.3 RetinaNet

RetinaNet is considered one of the top one-stage object detection models, particularly effective for detecting dense and small-scale objects. Therefore, it has gained popularity as a model for object detection applied to aerial and satellite imagery [10].

RetinaNet enables efficient single-stage object detection through four core components. It uses a ResNet backbone to generate feature maps at various resolutions, which feed into a top-down pathway that forms a Feature Pyramid Network (FPN). This FPN combines low-resolution, semantically strong features with high-resolution, spatially detailed features for robust multi-scale detection. At each FPN level, there are two subnetworks: one for class probability predictions (classification subnetwork) and another for bounding-box offsets (regression subnetwork) [10]. A key innovation is the use of Focal Loss, which addresses class imbalance by focusing training on challenging examples, allowing RetinaNet to achieve high accuracy while maintaining the speed of a one-stage detector [11].

2.4 EfficientDet

EfficientDet [12] is an object detection architecture that prioritizes efficiency and scalability. It builds upon the EfficientNet [13] backbone and introduces two key innovations: the Weighted Bi-directional Feature Pyramid Network (BiFPN) and a compound scaling method.

The BiFPN allows for easy and fast multi-scale feature fusion by introducing learnable weights to learn the importance of different input features significantly. Unlike traditional FPNs, it incorporates top-down and bottom-up pathways with cross-scale connections to enable more efficient information flow. Furthermore, EfficientDet utilizes a compound scaling method that uniformly scales the resolution, depth, and width for all backbone, feature network, and box/class prediction networks, ensuring optimal performance across a wide range of resource constraints.

2.5 YOLO (You Only Look Once)

YOLO [9] fundamentally changed the approach to object detection by framing it as a single regression problem, rather than a classification task applied to region proposals. Unlike two-stage detectors like Faster R-CNN, which first generate potential regions and then classify them, YOLO processes the full image in a single pass through the neural network.

The architecture divides the input image into a grid. If the centre of an object falls into a grid cell, that cell is responsible for detecting the object. Each grid cell predicts bounding boxes and confidence scores for those boxes, along with class probabilities. This unified architecture allows for extremely fast inference speeds suitable for real-time applications. While early versions (YOLOv1-v3) sometimes struggled with small objects compared to region-based methods, modern iterations (such as the YOLOv8-v12 models used in this project) have incorporated feature pyramids and advanced augmentation techniques to significantly close this gap.

3 Data Preparation

To prepare the dataset for training, the following steps were taken:

1. The images were collected manually from diverse locations on the island, and at different times of the day. The goal was to capture a solid variety of sign conditions, angles, and lighting scenarios.

2. The collected images were then cleaned to remove any duplicates, after which, any recognizable faces or licence plates were obscured to ensure privacy. This was done both manually and using automated tools that can be found in the 'Annotation Process' folder of the GitHub repository [1] (all automated outputs were manually verified before moving onto the next step).

3. The images were then annotated manually using LabelStudio as instructed.

4. Once all images were annotated by each member, the provided `MemberMerger.py` was used to merge every team member's individual annotation JSON files and image ZIP files into one single merged dataset. The output from this step (`merged_input.json` and `merged_images.zip`) was then uploaded to the GitHub repository and can be found in the subdirectory `Dataset/`.

5. The merged Label Studio JSON annotations were then converted into a COCO-formatted dataset by using the provided `LS2COCO.ipynb`. This step produced five COCO-format annotation files, which were uploaded to the GitHub repository and can be found in the `Dataset/COCO-Format Annotations/` subdirectory.

6. The dataset was then split into Train/Validation/Test subsets by using `LS2COCO.ipynb`. A 70%/15%/15% split was chosen to maximize the training data while still retaining sufficient validation and test sets, given the small dataset size. This resulted in a dataset of 485 train images, 111 validation images, and 119 test images.

7. The dataset splitting process was repeated for all attributes and for both YOLO and COCO-based formats.

8. The split dataset was then zipped and uploaded to the shared repository folder (`Dataset/Train-Test-Val Split/`) for easy access by all group members.

3.1 Dataset Class Distribution

Each team member tried to collect a balanced amount of images per class as possible. However, due to some types of signs being less common than others, imbalances appeared in the final dataset.

The class distribution for the Sign Type (as illustrated in Figure 1) is moderately imbalanced, with 'No Entry (One Way)' signs being the most frequent class (26.19% of all signs). 'Pedestrian Crossing' (19.17%), 'Stop' (18.22%), and 'Blind-Spot Mirror' (16.6%) have similar representation. However, 'Roundabout Ahead' (11.57%) and 'No Through Road' (8.25%) signs appear less frequently.

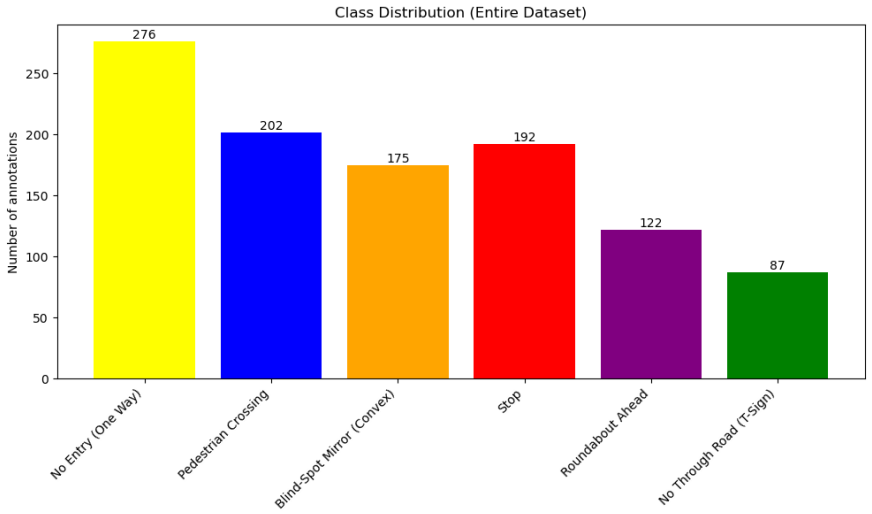


Figure 1: Class Distribution of Sign Types

The Sign Shape Type attribute shows a noticeable class imbalance. Circular signs made up nearly half of the dataset (48.58%), followed by square (20.49%) and octagonal (17.55%) signs, which are moderately represented. Triangular signs are significantly less frequent (10.63%), while the damaged class is severely underrepresented (2.75%). This imbalance reflects the real-world frequency of the traffic signs that were encountered during image collection, where circular signs (such as No Entry signs, which were also the most common sign type) were more commonly observed than other sign shapes.

A very large imbalance could also be seen in the Mounting Type attribute, with 87% of all signs being pole-mounted, while only 13% were wall-mounted. This is due to the fact that the majority of signs found across the island were pole-mounted, and wall-mounted signs were much rarer. Similarly, the class distribution for the Sign Condition attribute was also greatly imbalanced. Out of all signs, only 3.8% were labelled as heavily damaged, while 67.08% of signs were in good condition, and 29.13% were weathered.

The Viewing Angle attribute was the most balanced out of all attributes, due to the fact that each sign was photographed from all three viewing angles. 39.75% of all signs were captured from the front, 30.46% from the back, and 29.79% from the side. Perfect balance was not achieved due to signs that also appeared in the background, which were captured from different angles.

4 Implementation of the object detectors

4.1 COCO-based Detectors

The implementation of the different COCO-based detectors varied slightly between models due to their unique architectures and requirements, and having been implemented by different group members. In general, the aim was to match the outputs of the YOLO-based detectors in terms of training performance metrics. The main metric we aimed to optimize was the mAP at IoU=0.5 and at 0.5:0.95. The following COCO-based detectors were implemented:

4.1.1 EfficientDet

The EfficientDet architecture, utilizing the `tf_efficientdet_d0` backbone, was chosen for its optimal trade-off between detection accuracy and resource efficiency. This model integrates an EfficientNet backbone with a BiFPN feature fusion network, allowing it to learn multi-scale feature representations effectively. Pre-trained on ImageNet, the model was fine-tuned to detect the six specific road sign classes in the dataset.

Dataset and Preprocessing

A custom `EfficientDetDataset` class handles data loading and transformation. Images are read via OpenCV, converted to RGB, and resized to 512×512 pixels while preserving aspect ratio through padding. Annotations are parsed from the COCO-format JSON, with bounding boxes normalized to the model's required format. Normalization is applied using ImageNet statistics ($\mu = [0.485, 0.456, 0.406]$, $\sigma = [0.229, 0.224, 0.225]$).

Augmentation

To mitigate overfitting on the small dataset, the `Albumentations` library was employed. Augmentations include horizontal flipping ($p = 0.5$), shift-scale-rotate transformations, random brightness/contrast adjustments, and RGB shift, ensuring the model is robust to varying environmental conditions.

Training Setup

Training was conducted using the `DetBenchTrain` module with the AdamW optimizer ($lr = 1e-4$) and a batch size of 4 to fit within GPU constraints. A class balancing strategy was implemented where minority classes were oversampled to match the frequency of the majority class, preventing bias toward frequent sign types. The model heads were fine-tuned while keeping the backbone frozen initially to stabilize convergence.

4.1.2 Faster R-CNN

The Faster R-CNN implementation was based on the pre-trained `fasterrcnn_resnet50_fpn_v2` model from torchvision, using the FasterRCNN_ResNet50_FPN_V2_Weights.DEFAULT weights as a starting point. The model was fine-tuned on the custom traffic sign dataset with 6 sign classes plus a background class.

Model Architecture

The base model utilizes a ResNet-50 backbone with Feature Pyramid Network (FPN) for multi-scale feature extraction. The region of interest (RoI) heads were modified to accommodate the custom number of classes by replacing the box predictor with a FastRCNNPredictor configured for 7 output classes.

Dataset Configuration

The training and validation datasets were loaded using PyTorch's CocoDetection class with images transformed to tensors. A custom anchor generator was configured with five anchor sizes (16, 32, 64, 128, 256 pixels) and three aspect ratios (0.5, 1.0, 2.0) to better detect signs of varying sizes. The training used a batch size of 6 with pin memory enabled for faster GPU data transfer.

Training Configuration

The model was trained for up to 100 epochs with the following hyperparameters:

- Learning rate: 0.005
- Momentum: 0.9
- Weight decay: 0.0005
- Optimizer: SGD
- Learning rate scheduler: StepLR (step size=25, gamma=0.1)
- Early stopping patience: 10 epochs

Loss Components

The total training loss was composed of four components: box regression loss, RPN box regression loss, classifier loss, and objectness loss. For logging purposes and comparison with YOLO results, these were grouped into box loss (loss_box_reg + loss_rpn_box_reg) and classification loss (loss_classifier + loss_objectness).

Evaluation Metrics

Validation was performed using COCO evaluation metrics, specifically tracking mAP@IoU=0.50 and mAP@IoU=0.50:0.95. The model's predictions were converted to COCO result format and evaluated using the pycocotools library. Early stopping was implemented based on mAP@IoU=0.50:0.95 to prevent overfitting.

Logging and Monitoring

Training progress was monitored using TensorBoard for real-time visualization of loss curves and metrics. Additionally, a CSV file in YOLO-compatible format was generated containing epoch-wise metrics including training/validation losses, mAP scores, recall, and learning rates. The model was saved periodically every 10 epochs, with the best performing model (based on validation mAP) saved separately.

The implementation was executed on hardware consisting of an AMD Ryzen 9 5950X CPU (16 cores/32 threads), 64 GB RAM, and an NVIDIA GeForce RTX 5070 Ti GPU with 16 GB VRAM, using CUDA 13.1 for acceleration.

4.1.3 RetinaNet

The RetinaNet model implemented in this study uses the `retinanet_resnet50_fpn_v2` architecture provided by the Torchvision library. This architecture uses a ResNet-50 backbone integrated with a Feature Pyramid Network (FPN) to extract multi-scale feature representations. To adapt our specific traffic sign dataset, the default classification head was replaced with a custom `RetinaNetClassificationHead`. This modification enables the model to output predictions for seven distinct classes. The model is initialised via a specialised `build_retinanet()` function, which loads the pre-trained weights for the FPN backbone and configures the classification head according to the required output dimensions.

Dataset Preparation

Training and validation data were managed using a custom `CocoDetectionDataset` class, designed to parse COCO-formatted JSON annotations and apply image transformations defined within the `get_transform()` function. Before training, the dataset directory was prepared using the `extract_images_zip()` utility, followed by annotation correction using `fix_coco_json()` to ensure compatibility with the COCO API. Both the training and validation data loaders were configured with a batch size of 2. To handle different lengths of target dictionaries and varying numbers of bounding boxes in object detection, we created a custom `collate_fn` for batching.

Training Setup

The RetinaNet model was trained for up to 100 epochs using the AdamW optimizer (learning rate: 1e-4, weight decay: 0.01). A StepLR scheduler reduced the learning rate every 25 epochs, while gradient clipping with a maximum norm of 0.1 was enabled to prevent exploding gradients, which can occur when training large dense prediction heads such as RetinaNet's classification subnetwork. The model computed and logged classification and bounding-box regression losses to TensorBoard. Validation loss was evaluated in training mode, and metrics were calculated using the COCOeval API. Checkpoints were saved every 10 epochs, the best model was tracked based on mAP@0.50:0.95, and early stopping with a 20-epoch patience was used to optimize computation.

Evaluation and Metric Computation

Model evaluation was performed using COCO-style detection metrics through the `COCOeval` API. During the validation phase, the model was switched to evaluation mode (`model.eval()`) to disable dropout and update batch normalisation. Predictions were generated for each image, resulting in outputs that included bounding boxes, class labels, and confidence scores. These outputs were then reformatted into COCO-style detection entries. Each detection was matched to its corresponding `image_id` from the validation JSON annotations to ensure accurate metric computation.

Logging and Monitoring

Throughout the training, detailed logs were performed using TensorBoard's SummaryWriter. Logged values included total loss, individual loss components, learning rate schedules, precision/recall metrics, and both mAP scores. Additionally, each epoch's metrics and losses were attached to a structured CSV file, allowing direct comparison with YOLO-based detectors and facilitating external analysis and plotting. Progress bars were used to visualize training and validation progress in real time, and epoch duration was recorded to monitor runtime efficiency.

4.1.4 RF-DETR

The RF-DETR implementation utilized the `rfdetr` library to facilitate with loading and training the pre-trained model. The model size used for the implementation in this project was the RF-DETR Base model. The base model was initialized using the built-in class `RFDETRBase()` provided by the `rfdetr` library. The model was then fine-tuned on the Maltese traffic signs dataset to be able to detect the six different traffic sign types.

Dataset Preparation

Before training, the dataset had to be organised and formatted into the structure expected by RF-DETR. RF-DETR expects the dataset to contain three subdirectories (named `train`, `valid`, and `test`) so each directory containing the images for that split was renamed as such. Each subdirectory needs to contain its own annotations file for that split, named `_annotations.coco.json`, so each file was renamed and placed in the corresponding subdirectory. The second preparatory step that had to be taken was adding `supercategory` to every category in all the annotation files. This had to be done because RF-DETR expects the `categories` entries to include the key `supercategory`, but these were not present in the original annotation files, and was resulting in a `KeyError`. One common supercategory name was chosen (`traffic_sign`) and added to each category. This ensured all traffic sign classes were grouped under a common parent category. Finally, the final step that had to be taken was to change the 1-based indexing of the category IDs in the original annotation files to 0-based IDs, as this was resulting in an index error. A preprocessing function was created to update all category IDs and corresponding annotations to prevent index errors during training.

Training Configuration

Training was performed using the built-in `train()` method from `rfdetr` with the following parameters. The maximum number of epochs was 100. Early stopping was enabled with patience of 20 epochs. A batch size of 4 was used along with a gradient accumulation of 4 steps, to maintain a total batch size of 16. This was the recommended batch size setup for training on smaller GPUs. The input image resolution was lowered slightly to 448×448 for efficiency. A learning rate of 1×10^{-4} was used. Exponential Moving Average (EMA) of weights was kept enabled, so that a model with EMA weights was also produced during training. This often improves results, and overall the EMA model was performing better during training than the base model. The final model checkpoint automatically chosen as the best model was the better one out of the EMA and non-EMA models.

Logging and Monitoring

TensorBoard logging was enabled to track and monitor loss and metrics over time while training. The main evaluation metrics tracked during training were mAP@IoU=0.50 and mAP@IoU=0.50:0.95. During analysis, it was observed that TensorBoard labeled the mAP@IoU=0.50:0.95 metric incorrectly as mAP@IoU=0.50:0.90. This issue was investigated by comparing the mAP@IoU=0.50:0.95 values printed in the training console output with the mAP@IoU=0.50:0.90 values recorded in TensorBoard. The numerical values matched exactly, confirming that the mAP@IoU=0.50:0.95 metric was being computed correctly and only the label in TensorBoard was incorrect. Therefore, for the RF-DETR implementation in this project, any TensorBoard metric labeled as mAP@IoU=0.50:0.90 should be interpreted as mAP@IoU=0.50:0.95. This issue affected only the metric naming in logs and did not impact training, evaluation, or model performance. Model checkpoints were saved periodically every 10 epochs, and the best performing checkpoint was automatically selected and saved as `checkpoint_best_total.pth`. Additionally, a `results.json` file was saved containing the final model evaluation metrics on the validation and test sets.

4.2 YOLO-based Detectors

The implementation of different YOLO versions (YOLOv8, YOLOv11, and YOLOv12) followed a consistent methodology across all group members, with variations primarily in model size selection and hardware-specific optimizations. This section describes the common approach and highlights significant deviations.

Common Implementation Framework

All YOLO implementations utilized the Ultralytics library and followed a standardized workflow consisting of dataset preparation, model training, and evaluation phases.

Dataset Preparation

The dataset preparation phase was consistent across all implementations. Images were extracted from a compressed archive (`images.zip`) into the YOLO dataset directory structure. The `data.yaml` configuration file was programmatically updated to reflect absolute paths for the training, validation, and test splits on each user's system. This ensured portability across different development environments while maintaining consistent dataset access patterns.

Model Selection and Training Configuration

Different YOLO versions and model sizes were employed:

- YOLOv8: Nano variant (`yolov8n.pt`) for baseline experiments
- YOLOv11: Small variant (`yolo11s.pt`) for balanced performance
- YOLOv12: Multiple variants tested (Large, Medium, Small) to evaluate size-performance tradeoffs

The standard training configuration shared across implementations included:

- Epochs: 100 with early stopping patience of 20
- Image size: 640×640 pixels (default)
- Device: Automatic GPU selection when available, CPU fallback otherwise
- Checkpoint saving: Every 10 epochs
- Task: Object detection (`task='detect'`)
- Project output: Organized under `runs/` directory

Significant Deviations

Hardware-Constrained Optimization

One implementation utilized reduced batch size (8) and image resolution (512×512) due to hardware memory limitations, demonstrating adaptive configuration for resource-constrained environments.

Advanced Optimizer Configuration

An alternative implementation experimented with the Adam optimizer instead of the default SGD, along with disk caching (`cache='disk'`) and increased worker threads (`workers=8`) to optimize I/O performance.

Automated Batch Sizing

The YOLOv12 experiments utilized automatic batch sizing (`batch=-1`) targeting 60% GPU utilization, allowing the framework to dynamically determine optimal batch sizes based on available VRAM.

Data Augmentation Experiments

Extended experiments with YOLOv12 included systematic comparison of default training versus augmented training with modified hyperparameters:

- Horizontal/vertical flipping disabled (`fliplr=0.0, flipud=0.0`) to preserve sign orientation
- Mosaic augmentation maintained at full strength (`mosaic=1.0`)
- Mixup augmentation introduced (`mixup=0.1`)
- Rotation augmentation enabled (`degrees=10.0`)

- Scale variation applied (`scale=0.5`)

These augmentation strategies were designed specifically for traffic sign detection, where orientation preservation is critical but lighting and scale variations are beneficial.

Monitoring and Logging

All implementations enabled TensorBoard logging for real-time training visualization. Training metrics including loss curves, mAP scores, precision, and recall were logged automatically to CSV files in YOLO's standard format, facilitating cross-model comparison and performance analysis.

5 Evaluation of the object detectors

5.1 COCO-based Detectors

5.1.1 EfficientDet

The EfficientDet model was evaluated using a classification-based approach on ground truth crops due to limited training resources and time constraints. The model achieved an overall validation accuracy of 44.12% after 30 epochs.

The per-class accuracy breakdown highlights significant performance variance:

- No Entry: 91.67% (Highest accuracy)
- Pedestrian Crossing: 66.67%
- Blind Spot Mirror: 34.62%
- Stop: 33.33%
- No Through Road: 0.00%
- Roundabout Ahead: 0.00%

These results indicate that while the model successfully learned features for distinct signs like 'No Entry', it struggled significantly with others, likely due to class imbalance, insufficient training data for those specific categories, or the limited training duration.

5.1.2 Faster R-CNN

The Faster R-CNN model, utilizing a ResNet-50 FPN V2 backbone, was trained for 58 epochs using the SGD optimizer with an initial learning rate of 0.005. The model was evaluated using standard COCO metrics on the full validation set.

The model achieved the following peak performance metrics:

- mAP@.50: 71.45%
- mAP@.50:.95: 59.46%
- Recall: 68.83%

Analysis of the results indicates strong localization capabilities, though a performance gap persists between large and small objects. As shown in Figure 2 (right), the bounding box accuracy (mAP@.50:.95) continued to improve until approximately epoch 46, even after the classification metrics began to plateau.

A notable observation in the training dynamics is the divergence in classification loss illustrated in Figure 2 (left). While the training classification loss continued to decrease, the validation classification loss exhibited a gradual upward trend after epoch 4. This phenomenon suggests a degree of overfitting, where the model becomes over-confident in the specific categorical features of the training set, leading to a penalty in cross-entropy loss on unseen validation data. However, since the mAP scores remained stable or improved during this period, the model's actual predictive accuracy remained robust, indicating that the overfitting was primarily restricted to loss confidence rather than categorical misclassification. The final combined validation loss settled at 0.2852.

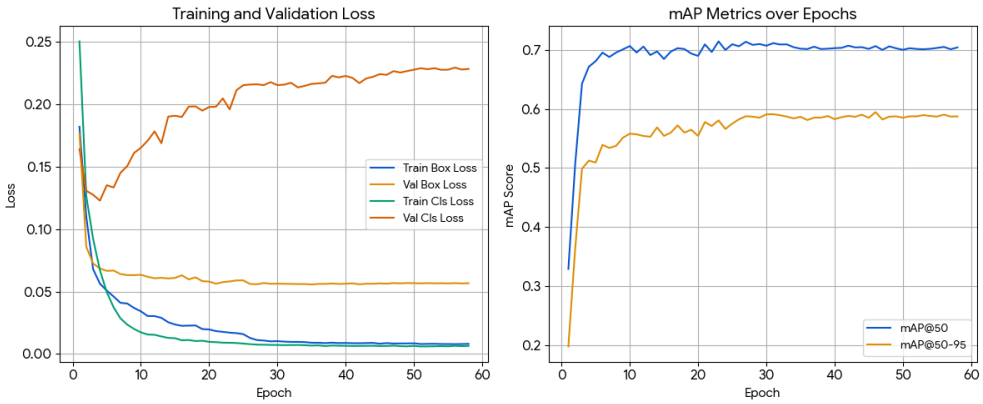


Figure 2: Faster R-CNN Training Performance Metrics.

5.1.3 RetinaNet

The model trained was trained for 70 epochs, since it had an early stopping patience of 20 epochs and a learning rate of 0.0001. The best-performing RetinaNet model was obtained at epoch 50, achieving the following validation results:

- mAP@IoU=0.50: 66.04%
- mAP@IoU=0.50:0.95: 54.09%
- Precision: 64.81%

The high mAP@0.50 indicates that the model is good at detecting traffic signs. However, the lower mAP@0.50:0.95 shows that it struggles with accurately locating signs when considering stricter criteria. The recall value reveals that although the model identifies most signs, it has trouble detecting some, particularly smaller or more distant ones.

As shown in Figure 3, the RetinaNet model displayed stable learning in the early training stages, with a consistent decrease in training loss. Validation loss initially dropped but started to rise after about 10–15 epochs, indicating overfitting. Despite this, detection metrics improved, with both mAP@0.50 and mAP@0.50:0.95 steadily increasing for the first 30–40 epochs before plateauing. The highest recorded values were around epoch 50, with mAP@0.50 stabilising at 0.64–0.66 and mAP@0.50:0.95 at 0.52–0.54. This shows that even with overfitting, the model refined its detection performance, especially in bounding-box quality across different IoU thresholds.

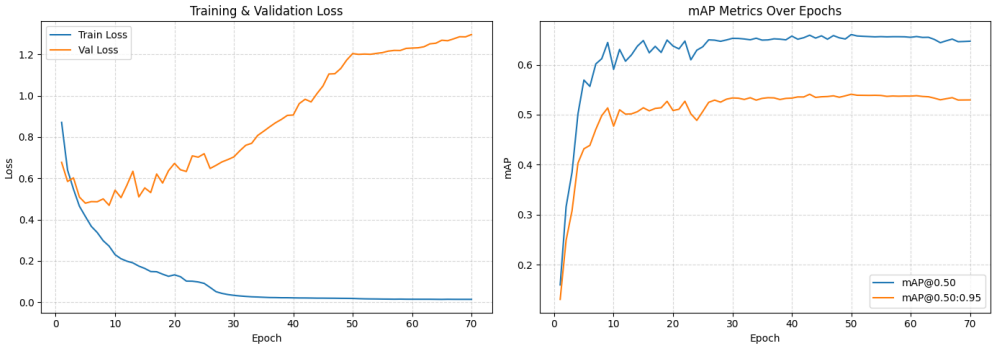


Figure 3: Retina Net Training Performance Metrics.

5.1.4 RF-DETR

The RF-DETR model was trained for a total of 56 epochs and performance was evaluated using standard object detection evaluation metrics, including mAP@IoU=0.50, mAP@IoU=0.50:0.95, precision, and recall. The goal of the evaluation was to measure how accurately the model could detect traffic signs in an image and localize them using bounding boxes.

The final best model achieved the following results on the validation set:

- mAP@IoU=0.50: 78.61%
- mAP@IoU=0.50:0.95: 64.23%
- Precision: 83.38%
- Recall: 68%

A mAP@IoU=0.50 of 78.61% indicates that the model performs well at correctly identifying the traffic signs. Although the model has good detection capability, the lower mAP@IoU=0.50:0.95 (64.23%) shows that the performance decreases when it comes to precise localization of the traffic signs in the image. The high precision (83.38%) suggests that most predicted detections correspond to real traffic signs, so the model produces few false positives. The recall of 68% indicates that although most traffic signs are detected, some are still missed.

These performance metrics were also computed for each separate class as shown in Table 1.

	Blind-Spot Mirror (Convex)	No Entry (One Way)	No Through Road (T-Sign)	Pedestrian Crossing	Roundabout Ahead	Stop
mAP@IoU=0.50	86.10%	76.73%	81.16%	62.50%	74.44%	90.72%
mAP@IoU=0.50:0.95	72.17%	65.08%	66.58%	45.17%	59.17%	77.23%
Precision	100%	87.5%	84.62%	44.83%	83.33%	100%
Recall	68%	68%	68%	68%	68%	68%

Table 1: Per-class Performance Metrics

Detection performance varies across traffic sign types, as shown in Table 1. Stop signs achieved the highest performance, indicating that the model is confident and accurate when detecting this sign. Blind-Spot Mirror (Convex) signs also performed strongly. Pedestrian Crossing signs showed the lowest performance, which suggests that this class is more difficult for the model. The equal recall values (68%) across classes shows that missed detections are distributed fairly evenly rather than concentrated in one single category.

Training progress was monitored using loss and detection metrics over epochs, as illustrated in Figure 4. Training loss showed a general downward trend and so did validation loss, suggesting that the model does not significantly overfit. Both mAP@0.50 and mAP@0.50:0.95 improved steadily during training. The EMA model consistently outperformed the base model, indicating that weight smoothing improved generalization.

Evaluation was also performed on the test set to see how accurately the model can correctly classify each sign. As illustrated in the confusion matrix in Figure 5, it can be seen that the Blind-Spot Mirror, No Entry, and Stop sign types were the most correctly detected. Some confusion occurs between signs such as Pedestrian Crossing with No Through Road and Roundabout Ahead. No Entry, Blind-Spot Mirror, and Pedestrian Crossing signs were also occasionally missed by the detector.

Additionally, visual inspection of a few of the predictions on the sample test images show that the model generally successfully detects multiple traffic signs in a single image and bounding boxes are generally well-aligned with the true sign boundaries. However, some failure cases were also observed, where signs were sometimes missed or misclassified, especially signs that appear smaller or further away in the distance, or were partially covered.

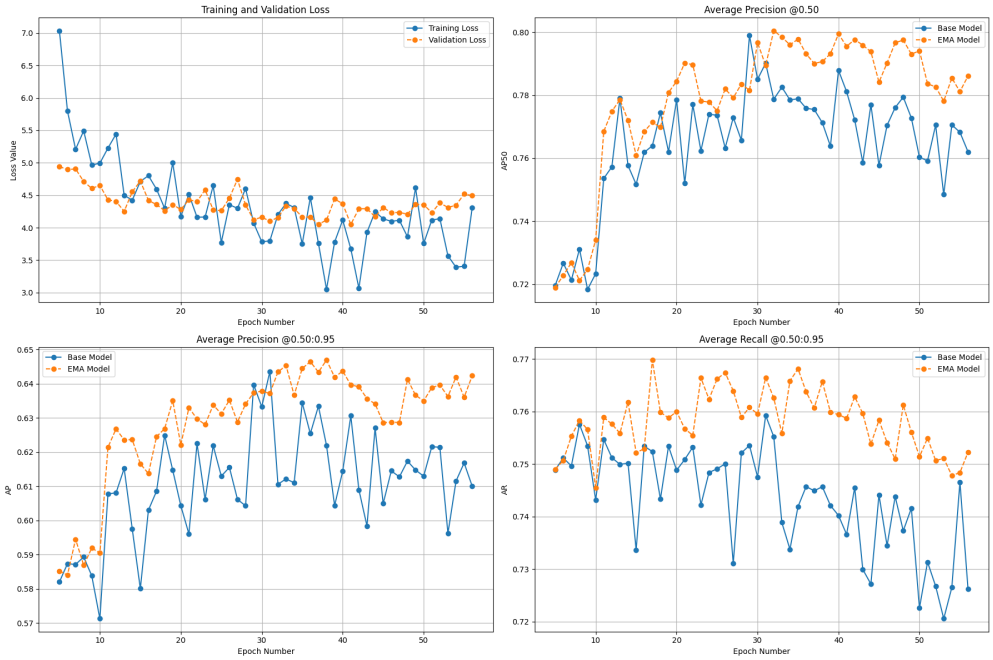


Figure 4: RF-DETR Training Metrics

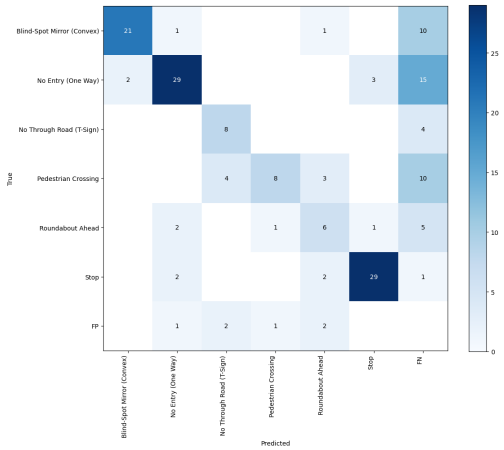


Figure 5: RF-DETR Confusion Matrix

5.2 YOLO-based Detectors

5.2.1 YOLOv8

Two YOLOv8 models were trained independently by different team members. Although both models were trained on the same traffic sign dataset, differences in training configuration and hyperparameters led to variations in performance. This section evaluates each model separately.

YOLOv8 - Joachim Grech

The YOLOv8 model, specifically the nano variant (`yolov8n`), was trained for 100 epochs using the Ultralytics framework and the Adam optimizer. This version of YOLO utilizes a C2f module and a decoupled head, improving the balance between speed and accuracy for object detection.

The model reached a peak performance with an overall mAP@.50 of 65.38% and an mAP@.50:.95 of 51.86%. The per-class mAP@.50 metrics provide further insight into its effectiveness:

- Blind-Spot Mirror (Convex): 78.2%
- No Entry (One Way): 66.3%
- No Through Road (T-Sign): 63.5%
- Pedestrian Crossing: 42.1%
- Roundabout Ahead: 58.2%
- Stop: 84.0%

YOLOv8 - Amy Spiteri

This model was trained for a total of 100 epochs and the best model checkpoint achieved the following performance metrics on the validation set:

- mAP@0.50: 67.98%
- mAP@0.75: 61.03%
- mAP@0.50:0.95: 54.88%

A mAP@0.50 of 67.98% shows that the model is able to correctly identify traffic signs under moderate IoU constraints, but the drop to 54.88% mAP@0.50:0.95 indicates that bounding box localization is not so accurate.

Precision and recall values for each separate class are also shown in Table 2. Blind-Spot Mirror and Stop signs achieved the highest precision, so predictions for these classes were usually correct. Pedestrian Crossing and No Through Road signs show lower recall, indicating that these signs were more frequently missed. The difference between precision and recall in several classes suggests the model is quite cautious and prefers fewer but more confident detections.

	Blind-Spot Mirror (Convex)	No Entry (One Way)	No Through Road (T-Sign)	Pedestrian Crossing	Roundabout Ahead	Stop
Precision	94.55%	78.93%	68.57%	72.54%	72.35%	85.61%
Recall	78.38%	54.84%	50%	47.37%	56.16%	80%

Table 2: YOLOv8 Per-class Performance Metrics

The mAP@0.50:0.95 curve over 100 epochs (as illustrated in Figure 6) shows rapid improvement during early training epochs and then stabilization after around epoch 40.

5.2.2 YOLOv11

The YOLOv11 model underwent training for 100 epochs utilizing the Ultralytics framework. As a high-performance one-stage detector, YOLOv11 adopts a three-stage backbone–neck–head architecture with improved feature fusion, designed to balance representational capacity with real-time inference requirements.

The model achieved the following peak performance metrics on the validation set:

- mAP@.50: 65.44%
- mAP@.50:.95: 55.12%

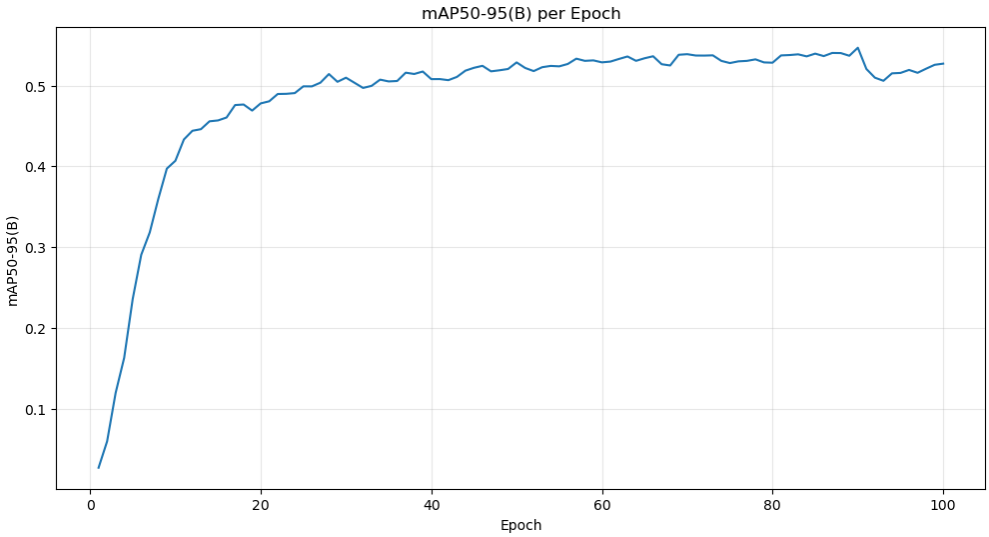


Figure 6: YOLOv8 mAP@0.50:0.95 per Epoch

- Precision (mean over classes): 74.62%
- Recall (mean over classes): 58.71%

As illustrated in Figure 7, the training results demonstrate an efficient and stable learning progression. The model achieved over 50% mAP within the initial epochs and continued to improve steadily throughout training. The validation classification loss (L_{val_cls}) stabilised at roughly 0.686, indicating stable generalisation. The YOLOv11 architecture, enhanced by DFL and advanced augmentation, effectively reduces categorical overfitting in this dataset.

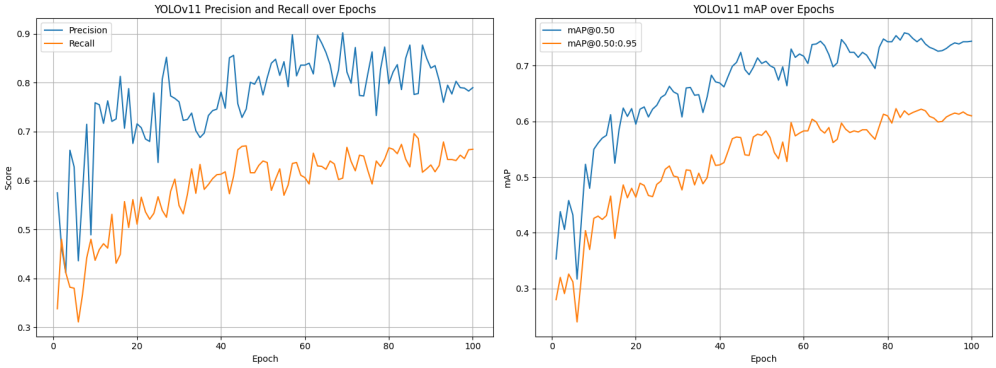


Figure 7: YOLOv11 Training Metrics.

5.2.3 YOLOv12

The YOLOv12 model was trained for 100 epochs using the Ultralytics framework. As a state-of-the-art one-stage detector, YOLOv12 utilizes an optimized backbone and neck architecture designed to maximize feature extraction efficiency while maintaining high inference speeds.

The model achieved the following peak performance metrics on the validation set:

- mAP@.50: 81.72%
- mAP@.50:.95: 70.45%

- Precision: 90.02%
- Recall: 79.78%

Analysis of the training results in Figure 8 shows a highly efficient learning curve, with the model surpassing 50% mAP within the first few epochs. The model maintained a consistent, stable trajectory, for the validation classification loss (L_{val_cls}) eventually settling at 0.686. This suggests that the YOLOv12 architecture, likely aided by Distribution Focal Loss (DFL) and advanced data augmentation, is resilient to categorical overfitting on this specific dataset.

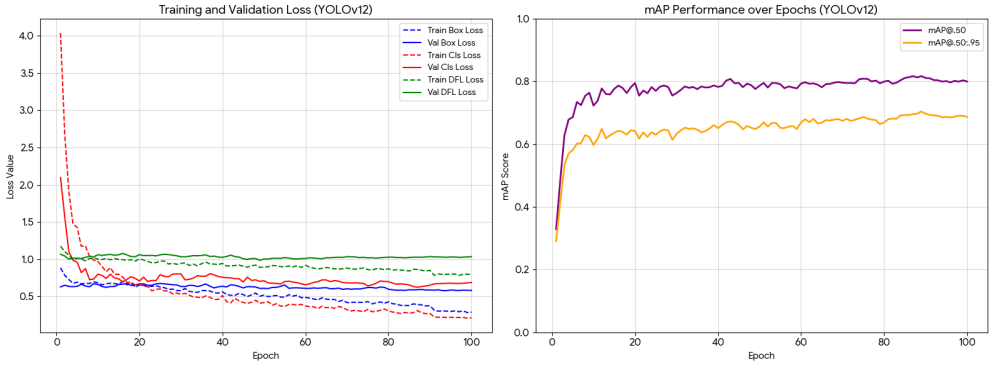
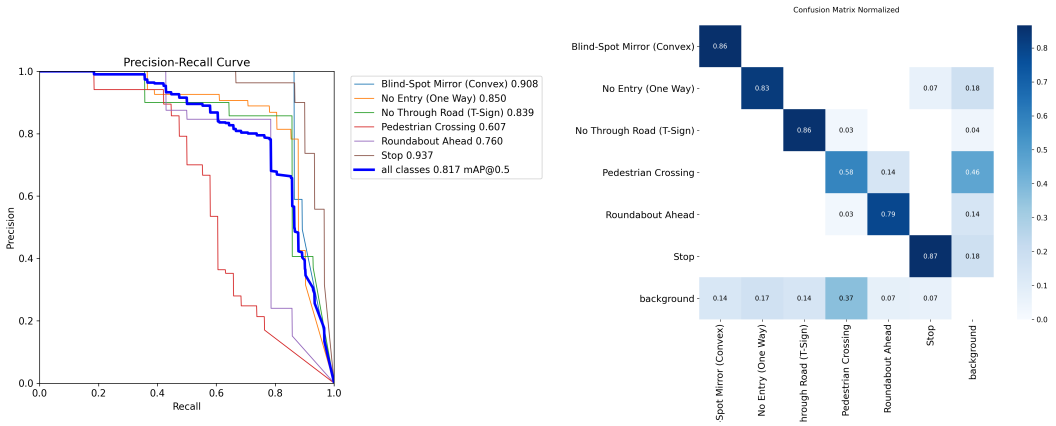


Figure 8: YOLOv12 Training Metrics.

Upon inspection of the precision-recall curve (Figure 9a), it became evident that the model achieved similar performance across all classes except 'Pedestrian Crossing'. This is further corroborated by the confusion matrix (Figure 9b), which shows a higher incidence of misclassifications for this class compared to others. The 'Pedestrian Crossing' signs were often confused with 'Roundabout Ahead' and 'No Through Road (T-Sign)' signs, but mostly with the background class. This suggests that the model struggled to distinguish 'Pedestrian Crossing' signs from other similar sign types, potentially due to their visual similarities or insufficient representation in the training data, which is a surprising outcome given that 'Pedestrian Crossing' signs were the second most common sign type in the dataset.



(a) Precision-Recall Curve

(b) Confusion Matrix

Figure 9: Precision-Recall Curve and Confusion Matrix for YOLOv12

5.3 Comparison between the different detectors

During training, each team member was instructed to log their model's performance. To evaluate our object detection models, we used mean Average Precision (mAP), which is the standard performance metric in

modern object detection benchmarks such as COCO. mAP measures how well a model both detects objects and localizes them accurately within an image. Specifically, the two metrics we decided to focus on were mAP@50 and mAP@50:95. The mAP@50 metric uses a single IoU threshold of 0.50. A detection is counted as correct if the predicted bounding box overlaps the ground-truth box by at least 50%. This metric is more lenient, and focuses more on whether the object was detected than on extremely precise localization. The mAP@50:95 metric is stricter and more comprehensive. Instead of using only one IoU threshold, it uses multiple IoU thresholds from 0.50 to 0.95. This means that a higher value indicates that the model is not just good at detecting objects, but also good at placing bounding boxes very accurately. As a result, mAP@50:95 is typically lower than mAP@50 and better reflects overall localization quality.

In order to compare the performance of all the trained detectors, a robust script was required since, although all team members were using the same dataset, and aiming to capture the same metrics, slight variations in the implementation methods meant that the outputs required some 'massaging' to be comparable. A brief outline of the steps taken to achieve this is provided below, while the full implementation can be found in the 2c notebook.

1. Each model's results were saved and logged after every epoch during training. These logs were written to an events file for real-time monitoring of training progress using TensorBoard.

2. After each member completed training their specific models, the events files were collected and organized into a shared folder.

3. In the 2c notebook, the events files were parsed and the specific results of each model (mAP@50, mAP@50:95) were plotted (shown in Figure 10).

4. After noticing many duplicate/incomplete results in the initial plots, the following heuristics were applied to clean the data:

   - Remove duplicate results.
   - Merge results from the same runs that were split across multiple events files.
   - Disregard plots with missing metrics.
   - Only keep the single best / longest run for each model.

5. The cleaned results were then re-plotted to produce the final comparison graphs (shown in Figure 11 in Section 5.3.1).

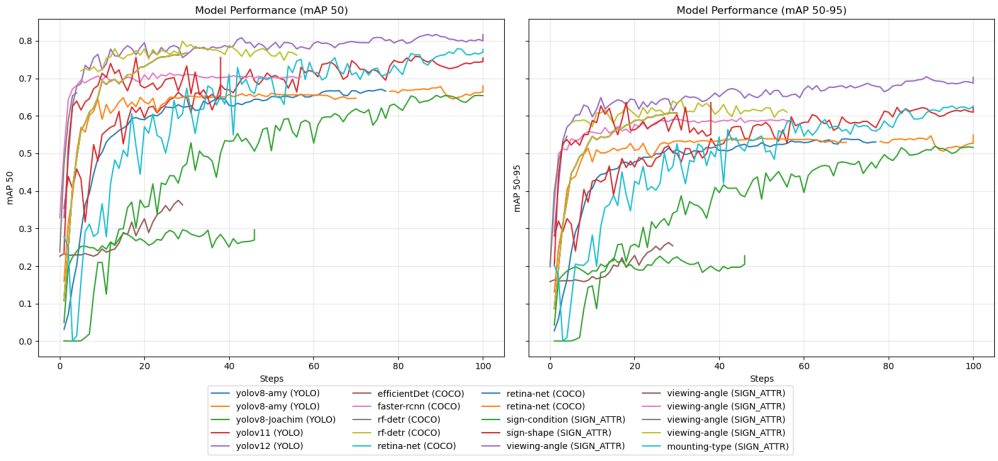


Figure 10: Comparison of mAP across different detectors

5.3.1 All Detectors together

The performance of all implemented models was aggregated to compare their learning trajectories and final accuracy, as illustrated in Figure 11. The evaluation focused on two primary metrics: mAP@50 and mAP@50-95.

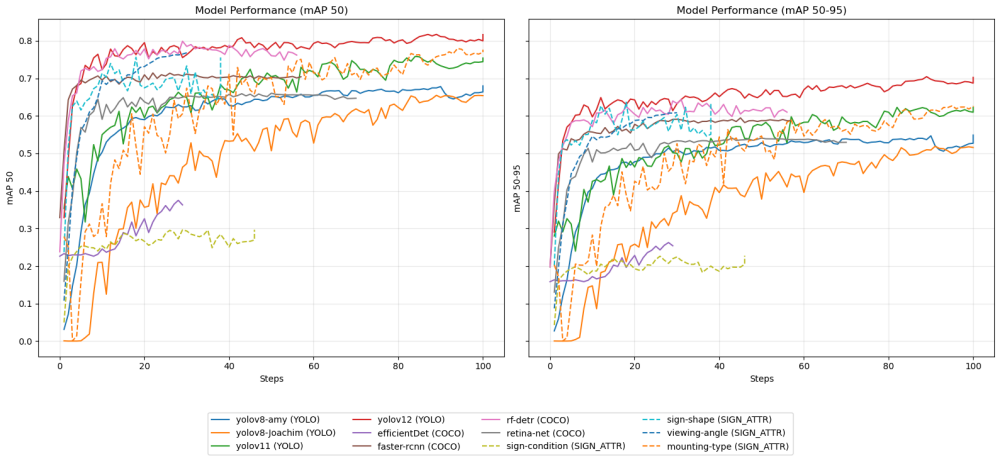


Figure 11: Comparison of mAP across after fixing

As seen in the results, **YOLOv12** (red line) emerged as the superior model, achieving the highest peak performance with an mAP@50 exceeding 0.80 and an mAP@50-95 reaching approximately 0.70. It demonstrated rapid convergence, stabilizing after roughly 40 steps. **Faster R-CNN** (brown line) and **rf-detr** (pink line) followed closely, maintaining competitive mAP@50 scores around $0.75 - 0.78$. These results suggest that the two-stage and transformer-based architectures are highly effective for this dataset's specific traffic sign features.

The detectors trained on specific sign attributes (prefixed with `SIGN_ATTR`) showed varying degrees of success:

- **Sign-shape** and **Viewing-angle** performed robustly, aligning with the mid-tier YOLO models with mAP@50 scores between 0.65 and 0.75.
- **Mounting-type** (orange line) exhibited a much slower learning curve, likely due to the extreme class imbalance (87% pole-mounted) noted in the data preparation section. It required the full 100 steps to reach an mAP@50 of 0.65.
- **Sign-condition** (light green line) was the worst performer among the attribute detectors, plateauing early at an mAP@50 of roughly 0.30. This is directly attributable to the severe underrepresentation of damaged signs (3.8%) in the dataset.

While **YOLOv8-amy** and **YOLOv11** showed stable, consistent growth, **EfficientDet** (purple line) failed to scale effectively, terminating early with significantly lower accuracy. The volatility observed in **YOLOv8-joachim** suggests that hardware-constrained optimizations (reduced resolution and batch size) introduced noise into the training process, though the model eventually reached a respectable mAP@50 of 0.75.

5.3.2 COCO-based vs YOLO-based Detectors

5.3.3 Sign Attribute Comparison

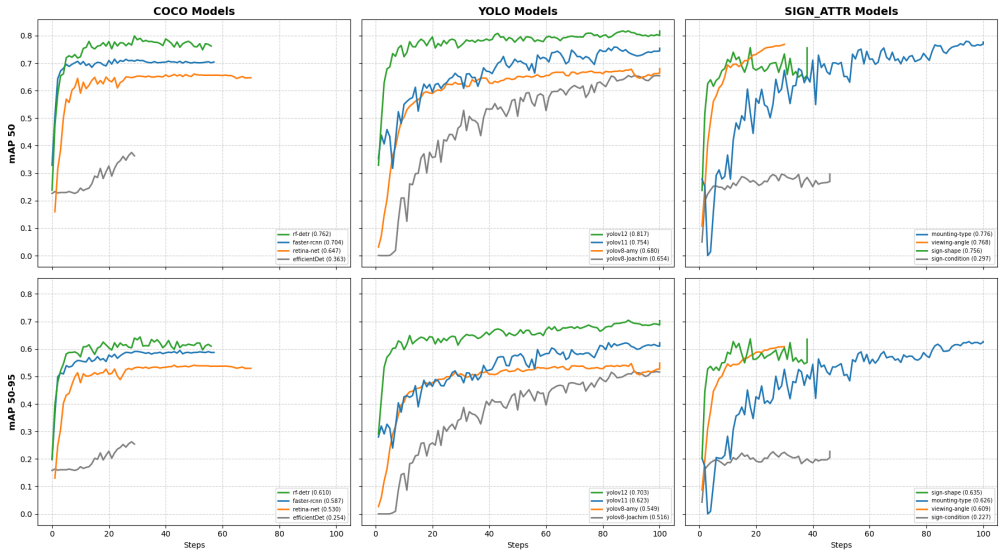


Figure 12: Comparison of Inference Times across different detectors

6 References and List of Resources Used

[1] A. Spiteri, D. Pace, E. R. Debrincat, and J. Grech, "Annotation Process Tools," GitHub repository, spiteriamy/ari3129-cv-group-project, Jan. 2026. [Online]. Available: `https://github.com/spiteriamy/ari3129-cv-group-project/tree/main/Annotation%20Process`

[2] S. Ren, K. He, R. Girshick, and J. Sun, "Faster R-CNN: Towards real-time object detection with region proposal networks," *IEEE Transactions on Pattern Analysis and Machine Intelligence*, vol. 39, no. 6, pp. 1137–1149, Jun. 2017.

[3] R. Girshick, "Fast R-CNN," in *Proceedings of the IEEE International Conference on Computer Vision (ICCV)*, Santiago, Chile, 2015, pp. 1440–1448.

[4] R. Girshick, J. Donahue, T. Darrell, and J. Malik, "Rich feature hierarchies for accurate object detection and semantic segmentation," in *Proceedings of the IEEE Conference on Computer Vision and Pattern Recognition (CVPR)*, Columbus, OH, USA, 2014, pp. 580–587.

[5] K. He, X. Zhang, S. Ren, and J. Sun, "Deep residual learning for image recognition," in *Proceedings of the IEEE Conference on Computer Vision and Pattern Recognition (CVPR)*, Las Vegas, NV, USA, 2016, pp. 770–778.

[6] I. Robinson, P. Robicheaux, M. Popov, D. Ramanan, and N. Peri, "RF-DETR: Neural Architecture Search for Real-Time Detection Transformers," arXiv preprint arXiv:2511.09554, 2025. [Online]. Available: https://arxiv.org/abs/2511.09554

[7] N. Carion, F. Massa, G. Synnaeve, N. Usunier, A. Kirillov, and S. Zagoruyko, "End-to-End Object Detection with Transformers," in *Computer Vision – ECCV 2020*, A. Vedaldi, H. Bischof, T. Brox, and JM. Frahm, Eds. Cham, Switzerland: Springer, 2020, pp. 213–229.

[8] M. Oquab et al., "DINOv2: Learning Robust Visual Features without Supervision," arXiv preprint arXiv:2304.07193, 2024. [Online]. Available: https://arxiv.org/abs/2304.07193

[9] J. Redmon, S. Divvala, R. Girshick, and A. Farhadi, "You Only Look Once: Unified, Real-Time Object Detection," in *Proceedings of the IEEE Conference on Computer Vision and Pattern Recognition (CVPR)*, Las Vegas, NV, USA, 2016, pp. 779–788.

[10] T. Y. Lin, P. Goyal, R. Girshick, K. He, and P. Dollár, "Focal Loss for Dense Object Detection," in *Proceedings of the IEEE International Conference on Computer Vision (ICCV)*, Venice, Italy, 2017, pp. 2980–2988.

[11] Viso.ai, "RetinaNet: The Definitive Guide (2025)," *Viso.ai Blog*, 2022. [Online]. Available: `https://viso.ai/deep-learning/retinanet/`. [Accessed: 30-Jan-2025].

[12] M. Tan, R. Pang, and Q. V. Le, "EfficientDet: Scalable and Efficient Object Detection," in *Proceedings of the IEEE/CVF Conference on Computer Vision and Pattern Recognition (CVPR)*, Seattle, WA, USA, 2020, pp. 10781–10790.

[13] M. Tan and Q. V. Le, "EfficientNet: Rethinking Model Scaling for Convolutional Neural Networks," in *Proceedings of the 36th International Conference on Machine Learning (ICML)*, Long Beach, CA, USA, 2019, pp. 6105–6114.

[14] Google DeepMind, "Antigravity: Advanced Agentic Coding Assistant," 2026. [Online].